# B. M. S. COLLEGE OF ENGINEERING

*Autonomous Institute, Affiliated to VTU, Belagavi*

## DEPARTMENT OF MACHINE LEARNING

Program: B.E. In Artificial Intelligence and Machine Learning

**Academic Year: 2026 - 27, Session: July 2026 - Dec 2026**

# Natural Language Processing for LLM (24AM7PCLLM)

**Alternative Assessment Tool (AAT) Report**

**Topic: Comparative Evaluation of Prompt Engineering Techniques for Structured Clinical Information Extraction Using Local Large Language Models**

**Submitted by**



| | |
|---|---|
| Student Name:<br>Team Member 1:<br>Team Member 2: | **Nemagouder Geeta Ajit**<br>**Pallavi M R** |
| USN:<br>Team Member 1:<br>Team Member 2: | **1BM24AI412**<br>**1BM24AI413** |
| Semester & Section: | **7 D** |



**Valuation Report**

*(to be filled by faculty)*

| | |
|---|---|
| **Score:** | |
| **Comments:** | |
| **Faculty In-charge:** | Dr Monika Puttaramaiah |
| **Faculty Signature with date:** | |

# Table of Contents


**1. Introduction** 3
- **1.1 Background of Natural Language Processing** 3
- **1.2 Importance of the Selected NLP Topic** 3
- **1.3 Objective of the Study** 3
- **1.4 Scope of the Report** 4

**2. Selected NLP Topic Overview** 5
- **2.1 Description of the NLP Task** 5
- **2.2 Real-World Use Cases** 5
- **2.3 Relevance to the Syllabus and Industry** 5
- **2.4 Key Challenges in This NLP Task** 5

**3. Real-Time Application / API Identified** 7
- **3.1 Application or API Name** 7
- **3.2 Purpose and Functionality** 7
- **3.3 Architecture or Platform on Which It Is Built** 7
- **3.4 Input/Output Behavior** 9
- **3.5 Limitations and Practical Considerations** 9

**4. NLP Model / Architecture Study** 10
- **4.1 Theoretical Foundations of the Model** 10
- **4.2 Algorithm(s) Used** 10
- **4.3 Architecture Diagram / Layer-Wise Design** 10
- **4.4 Data Flow in the Model** 11
- **4.5 Mathematical Formulation** 12
- **4.6 Why This Model Suits the Selected NLP Task** 12

**5. Dataset and Preprocessing** 13
- **5.1 Dataset Description (Source, Size, Structure)** 13
- **5.2 Data Cleaning Steps** 13
- **5.3 Tokenization, Lemmatization, Stopwords Removal** 14
- **5.4 Train/Test Split Strategy** 14
- **5.5 Feature Engineering or Embedding Approach** 14

**6. Implementation Methodology** 15
- **6.1 Tools, Libraries, and Environment Used** 15
- **6.2 Model Implementation Steps** 15
- **6.3 API Integration** 16
- **6.4 Pseudocode / Workflow Diagram** 16
- **6.5 Hyperparameters and Training Configuration** 17

**7. Results and Evaluation** 18
- **7.1 Performance Metrics** 18
- **7.2 Visualization of Results** 18
- **7.3 Qualitative and Quantitative Analysis** 21
- **7.4 Error Analysis** 21
- **7.5 Comparison Between Approaches** 22

**8. Discussion** 23
- **8.1 Interpretation of Results** 23
- **8.2 Strengths and Weaknesses of the Model** 23
- **8.3 Challenges Encountered During Implementation** 23
- **8.4 Impact of Dataset Quality and Preprocessing** 23
- **8.5 How API or Application Enhances the NLP Task** 24

**9. Conclusion** 25
- **9.1 Summary of the Work Done** 25
- **9.2 Key Learnings and Insights** 25
- **9.3 Real-World Applicability** 25
- **9.4 Future Improvements or Extensions** 25

**10. References** 26

# Introduction

## 1.1 Background of Natural Language Processing

Natural Language Processing (NLP) is a core subfield of Artificial Intelligence situated at the intersection of computer science, computational linguistics and machine learning. Over roughly four decades it has moved through four broad paradigms:

- **Rule-based and symbolic systems (1980s to 1990s):** hand-crafted grammars, lexicons and heuristic rules. Precise, but brittle on colloquial or ungrammatical input.
- **Statistical and shallow machine learning (1990s to 2010s):** Hidden Markov Models, Conditional Random Fields (CRFs) and n-gram models with TF-IDF style features.
- **Deep learning and distributed representations (2013 to 2017):** Word2Vec and GloVe embeddings with LSTM and BiLSTM-CRF networks that learn features from data instead of relying on hand-built ones.
- **Transformers and pre-trained language models (2017 to present):** self-attention [3] led to bidirectional encoders such as BERT [6] and autoregressive decoders such as GPT-3 [1], LLaMA and Qwen.

Modern Large Language Models (LLMs) show emergent **in-context learning (ICL)** [1]. Instead of updating model weights for every downstream task, a practitioner steers a frozen model through **prompt engineering**: the task, the required output format and, optionally, worked examples are written directly into the input. For small models that run locally, the prompt is the main lever available for improving task performance, and it is the lever examined in this report.

## 1.2 Importance of the Selected NLP Topic

In healthcare, an often-cited estimate is that roughly 80% of clinical information sits in unstructured narrative text: outpatient encounter notes, emergency triage notes, discharge summaries and radiology reports. Narrative notes give clinicians expressive freedom, but they also create a data silo:

- Hospital databases and interoperability standards such as HL7 FHIR [7] expect structured records, not free text.
- Clinical decision support systems cannot raise drug-interaction or contraindication alerts from text they cannot parse.
- Cohort discovery for registries and clinical trials still depends on slow manual chart review.

**Structured Information Extraction (SIE)**, the conversion of a clinical narrative into a validated JSON object that follows a fixed schema, is therefore a high-value NLP task. It is also privacy-sensitive: regulations such as HIPAA (United States) and GDPR (European Union) restrict how Protected Health Information (PHI) may be sent to third-party cloud services. Running a small (1B to 3B parameter) instruction-tuned model on local hardware avoids that exposure, but such models are much less reliable at producing well-formed structured output. Identifying which prompt engineering technique closes this gap most effectively is the focus of this study.

## 1.3 Objective of the Study

The objectives of this Alternative Assessment Tool (AAT) project are to:

- Implement and benchmark five prompt engineering strategies: Direct Zero-Shot (baseline), Few-Shot In-Context Learning (ICL), Structured Chain-of-Thought (CoT), Schema-Constrained prompting, and Role-Conditioned Expert prompting with clinical guardrails.
- Deploy the pipeline on a fully local, privacy-preserving model (Qwen2.5:1.5B-Instruct served by Ollama) so that no patient text leaves the machine.
- Define a multi-criteria evaluation framework covering strict RFC 8259 JSON validity [14], schema key coverage, field-level F1 (demographics, diagnosis, severity, symptoms, medications, follow-up), a negation violation rate, and runtime telemetry (latency, token counts, tokens per second).
- Build an interactive Streamlit application for real-time extraction, side-by-side technique comparison, self-healing JSON repair and benchmark exploration.

- Draw evidence-based conclusions about which strategy gives the best balance of accuracy, structural reliability and speed on a small local model.

## 1.4 Scope of the Report

This report covers the NLP task and its real-world uses; the Ollama REST API and Streamlit application that form the working system; the architecture and mathematics of the Qwen2.5 transformer; the curated dataset of 20 clinical cases plus 2 held-out exemplars; the implementation of the pipeline; and the empirical comparison of the five prompt strategies.

> **Scope note.** The quantitative results in Section 7 come from a benchmark run on the **first 8 of the 20 cases** (8 cases × 5 strategies = 40 inference runs). Only one model, one decoding configuration and synthetic clinical text are studied, and the system is a research prototype, not a certified clinical tool. The sample size is restated wherever it affects interpretation.

# 2. Selected NLP Topic Overview

## 2.1 Description of the NLP Task

The selected task is **Structured Information Extraction (SIE)** from clinical narratives into a fixed JSON schema that mirrors the fields stored in an electronic health record and that could be mapped onto HL7 FHIR resources [7]. Classically it required a multi-stage pipeline:

1. **Named Entity Recognition (NER):** locate spans for diseases, symptoms, drugs, dosages and patient demographics.
2. **Assertion classification / negation detection:** decide whether each entity is present, absent or conditional [8].
3. **Relation extraction:** bind related mentions, for example the drug Metformin to its dose (500 mg) and frequency (twice daily).
4. **Slot filling and normalisation:** map mentions into a schema, optionally coded with UMLS, SNOMED CT or RxNorm.

With generative LLMs these stages collapse into one **in-context sequence transduction** step. Given a clinical note X and an instruction prompt I, the model produces the structured string Y token by token:

$$\hat{Y} = \underset{Y}{\operatorname{argmax}} \prod_{t=1}^{T} P(y_t \mid y_{<t}, X, I) \quad (1)$$

The output must satisfy two constraints at once: it must be syntactically valid JSON under RFC 8259 [14], and it must conform to the domain schema (keys and types in Table 3).

## 2.2 Real-World Use Cases

- **Automated EHR ingestion:** pre-populating electronic health record fields from dictated consultation notes or triage reports, removing hours of manual clerical entry.
- **Clinical decision support (CDSS):** turning physician notes into structured medication and condition records so rule engines can flag contraindications, duplicate therapy or dangerous interactions.
- **Medical coding and billing:** supporting ICD-10 and CPT code assignment, which speeds up reimbursement and reduces claim denials.
- **Clinical trial matching:** querying structured registries to find patients with a target phenotype instead of reviewing charts by hand.
- **Pharmacovigilance:** scanning inpatient progress notes for emerging adverse drug events.

## 2.3 Relevance to the Syllabus and Industry

The project maps directly onto the outcomes of *Natural Language Processing for LLM* (24AM7PCLLM):

- **LLM architecture:** causal decoder-only transformers, attention, RoPE and KV caching (Section 4).
- **Prompt engineering paradigms:** zero-shot, few-shot ICL, chain-of-thought, schema induction and role/guardrail prompting, compared empirically (Sections 6 and 7).
- **Evaluation methodology:** syntactic, set-based, token-level and negation-aware metrics that go beyond exact string matching (Section 7.1).
- **Local LLM deployment:** privacy-compliant, cost-free, on-premise inference that does not depend on third-party cloud APIs, which is a growing industry requirement in healthcare.

## 2.4 Key Challenges in This NLP Task

- **Syntactic fragility of small LLMs:** models of 1B to 3B parameters often emit unquoted numeric units (for example 198 mg/dL), trailing commas or conversational preambles, any of which breaks json.loads.
- **Clinical negation:** notes routinely state what is absent ("denies chest pain"). Naive extraction turns these into active symptoms, a clinically serious false positive [8].
- **Schema hallucination and key drift:** without explicit guidance, models invent their own keys and nesting, which prevents automated database ingestion.

- **Multi-attribute entity linking:** a medication is only useful when its name, dose and frequency are bound together in one object.
- **Latency versus reasoning:** techniques that elicit intermediate reasoning improve some fields but lengthen generation, which matters in real-time clinical workflows.

*Table 1: Challenges and the metric used to quantify each one*

| Challenge | How it is measured in this study |
|---|---|
| **Syntactic fragility** | Strict RFC 8259 validity (direct parse, before any repair) |
| **Clinical negation** | Negation violation rate (share of extracted symptoms that are negated in the note) |
| **Schema hallucination / key drift** | Schema key coverage (share of the 7 scored keys present) |
| **Multi-attribute entity linking** | Medications F1 (set-based matching of medication entries) |
| **Latency versus reasoning** | Mean latency per sample (s) and prompt / completion token counts |

# 3. Real-Time Application / API Identified

## 3.1 Application or API Name

1. **Inference backend:** the Ollama local REST API [15] (http://localhost:11434/api/generate), serving the quantized instruction-tuned model qwen2.5:1.5b.
2. **User-facing application:** an interactive web application built with Streamlit [16] (app.py), referred to in this report as the *Clinical Extraction Studio and Benchmark Explorer*.

## 3.2 Purpose and Functionality

**Ollama** manages the quantized GGUF weights in memory and exposes a simple HTTP endpoint that accepts a prompt plus sampling options (temperature, top-p, maximum new tokens) and returns the generated text. Nothing is streamed to external servers.

**The Streamlit application** wraps the whole research pipeline in four modules (Table 2). A sidebar indicator calls check_health() and shows whether the Ollama server and model are online.

*Table 2: Modules of the Streamlit application (app.py)*

| Module | Functionality |
|---|---|
| **Live Extraction Playground** | Run any of the five techniques on a benchmark case or on custom text, in single or side-by-side mode. Temperature slider (0.0 to 1.0, default 0.1). Metric cards show latency, tokens generated, tokens/s and JSON status (Valid, Repaired or Invalid). Shows the extracted profile (patient, diagnosis, severity, symptoms, medication table, follow-up) with tabs for parsed JSON, raw generation and the executed prompt. For benchmark cases it also scores the output against ground truth. |
| **Comparative Benchmark Dashboard** | Reads results/summary_metrics.csv (best values highlighted), shows the five plots in tabs, and offers a case-level error inspector with a technique filter backed by results/benchmark_results.json. |
| **Prompt Engineering Theory & Prompts** | Explains each paradigm and displays the exact prompt template used for it, rendered from TECHNIQUE_REGISTRY with a placeholder note. |
| **Project Report & Architecture** | Displays the system architecture and the target application (EHR ingestion with privacy-preserving local inference). |

Two engines support these modules. The **self-healing JSON parser** strips markdown fences, isolates the outermost braces and repairs common syntax errors, while still reporting whether the raw output was strictly valid. The **telemetry profiler** records wall-clock latency, prompt tokens, completion tokens and generation speed for every call.

## 3.3 Architecture or Platform on Which It Is Built

The system runs entirely on one workstation. Figure 1 shows how the user interfaces, prompt layer, inference server, parser and evaluator connect.

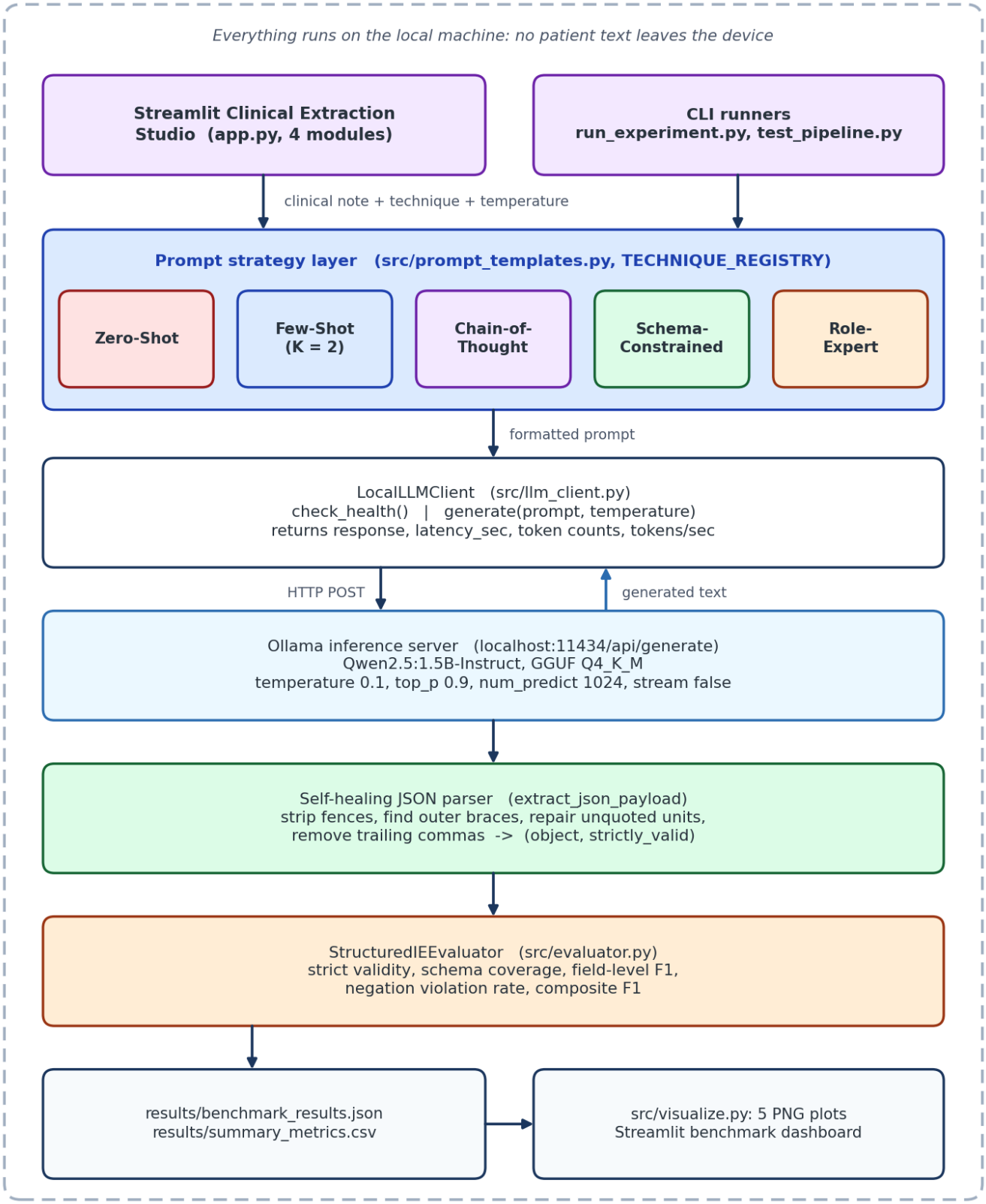



*Figure 1: End-to-end system architecture of the local extraction and evaluation pipeline*

- **Operating system:** Windows 11 workstation.
- **Runtime:** Python 3.11.
- **Inference server:** Ollama daemon on port 11434 serving qwen2.5:1.5b (GGUF, Q4_K_M quantization).
- **Front end:** Streamlit web app with Pandas tables and Matplotlib-generated plots.
- **Communication:** blocking HTTP POST requests (stream=False) through the Python requests library; one request is in flight at a time.

## 3.4 Input/Output Behavior

**Input:** free-form clinical narrative text (chief complaint, history, examination, medications, follow-up advice), the selected prompt strategy, and the sampling temperature (default 0.1).

**Output:** a JSON object following the schema in Table 3, together with run telemetry (latency_sec, prompt_tokens, completion_tokens, tokens_per_sec). Schema coverage is computed over the seven scored keys; the free-form vitals_and_labs dictionary is part of the target schema but is not scored.

*Table 3: Target JSON schema for clinical extraction*

| Field | Type | Description |
|---|---|---|
| **patient_age** | integer or null | Age in years |
| **patient_gender** | "Male" \| "Female" \| "Other" \| null | Patient gender |
| **primary_diagnosis** | string | Main diagnosis stated in the note |
| **severity_level** | "Mild" \| "Moderate" \| "Severe" \| "Critical" \| "Unknown" | Overall severity |
| **symptoms** | array of strings | Confirmed active symptoms only (negated findings excluded) |
| **medications** | array of objects | Each object has name, dosage and frequency |
| **vitals_and_labs** | dictionary | Free-form vital signs and laboratory values (not scored) |
| **follow_up_days** | integer or null | Days until the planned follow-up |

## 3.5 Limitations and Practical Considerations

- **Sequential processing:** the runner and the app issue one blocking request at a time, so throughput is bounded by single-stream generation speed (13 to 21 s per sample in this study, Table 10).
- **Hardware dependence:** the quantized model needs roughly 1 GB of memory plus KV cache. It runs on a CPU, but speed varies by machine, which is why tokens per second is logged for every run.
- **Context budget:** the Few-Shot and CoT prompts add about 950 prompt tokens (Table 11). With a 4096-token runtime context, long discharge summaries leave less room for the note and the generated JSON.
- **No hard validity guarantee:** generation is probabilistic, so even a well-prompted model can occasionally emit malformed or off-schema JSON. Output should be validated before it is stored.

# 4. NLP Model / Architecture Study

## 4.1 Theoretical Foundations of the Model

The model used is **Qwen2.5-1.5B-Instruct**, developed by the Qwen team at Alibaba Cloud [4]. It is an autoregressive, decoder-only transformer with 1.54 billion parameters, pre-trained on 18 trillion tokens of multilingual text, code and mathematical data, and then post-trained with supervised fine-tuning and preference optimisation, including Direct Preference Optimization (DPO) [13]. An autoregressive model factorises the probability of a sequence as a product of next-token probabilities, so structured output is produced one token at a time without any look-ahead; this is why a single early syntax error can invalidate the whole JSON object.

*Table 4: Key specifications of Qwen2.5-1.5B-Instruct*

| Property | Value |
|---|---|
| **Architecture** | Decoder-only transformer with causal attention |
| **Parameters** | 1.54 B (about 1.31 B excluding embeddings) |
| **Decoder layers** | 28 |
| **Hidden size (d_model)** | 1536 |
| **Attention** | Grouped Query Attention: 12 query heads, 2 key/value heads, head dimension 128 |
| **Feed-forward network** | SwiGLU, intermediate size 8960 |
| **Normalisation / positions** | Pre-layer RMSNorm / Rotary Position Embeddings (RoPE) |
| **Vocabulary** | About 151k byte-level BPE tokens |
| **Native context length** | 32,768 tokens (runtime context in this study: 4096 tokens) |
| **Deployed form** | GGUF, Q4_K_M quantization (about 1 GB) served by Ollama |

## 4.2 Algorithm(s) Used

- **Transformer decoder blocks:** 28 identical layers with pre-normalisation and residual connections.
- **RMSNorm [11]:** rescales activations by their root mean square without mean-centering, which is cheaper than LayerNorm and numerically stable.
- **Grouped Query Attention (GQA) [9]:** 12 query heads share 2 key/value heads, shrinking the KV cache while keeping most of the expressiveness of full multi-head attention.
- **SwiGLU [12]:** a gated feed-forward activation that replaces ReLU/GELU and improves gradient flow.
- **Rotary Position Embeddings (RoPE) [10]:** encode relative position by rotating query and key vectors, and support the long native context window.
- **GGUF Q4_K_M quantization:** stores weights in about 4 bits with block-wise scaling, cutting memory to roughly 1 GB with a small quality loss relative to FP16, which is what makes CPU inference practical.

## 4.3 Architecture Diagram / Layer-Wise Design

Figure 2 summarises the layer-wise design. Position information is injected inside every attention layer through RoPE rather than being added once to the input embeddings.

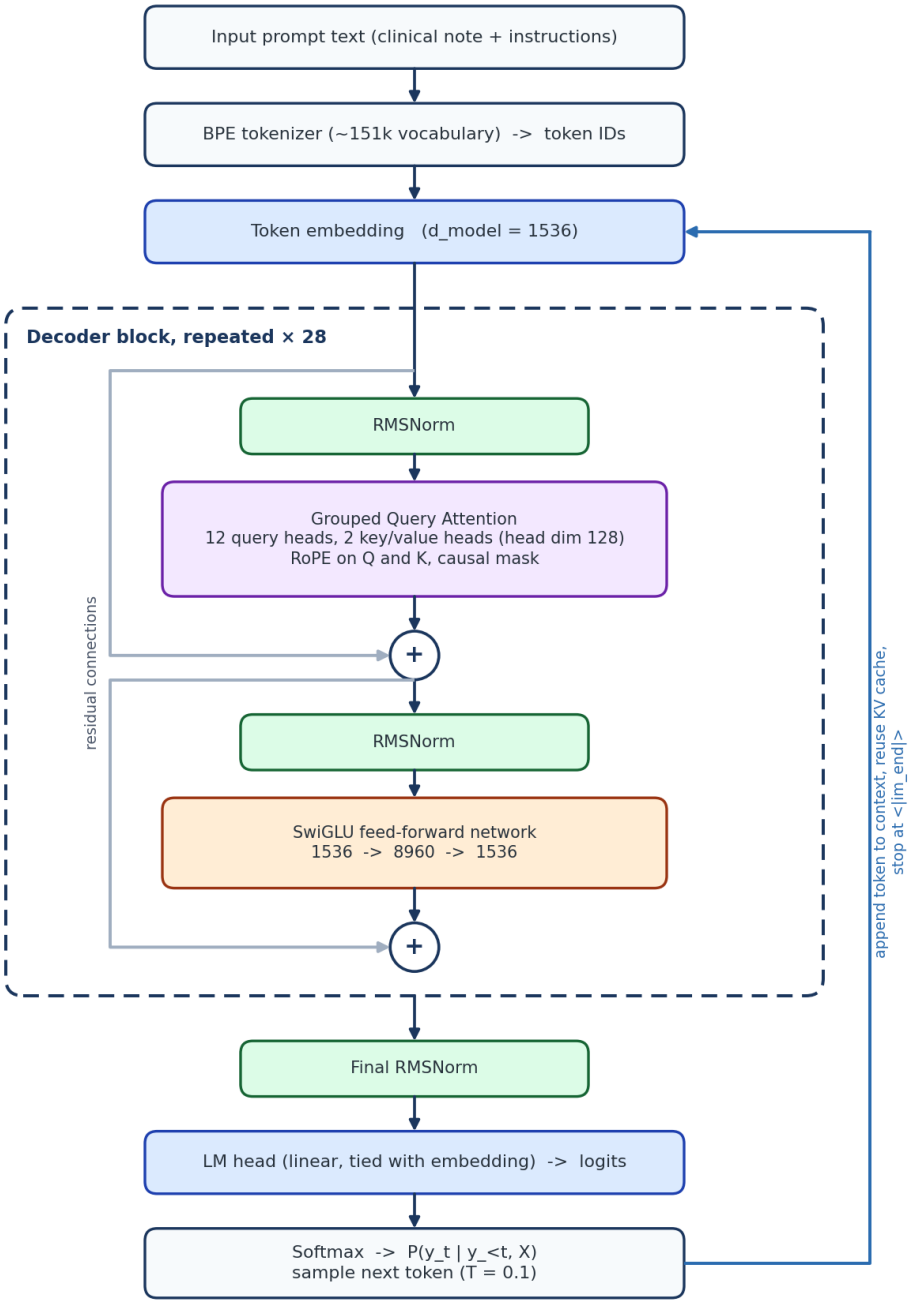



*Figure 2: Layer-wise design of the Qwen2.5-1.5B decoder-only transformer*

## 4.4 Data Flow in the Model

1. **Tokenization:** the prompt string (instructions plus clinical note) is converted into integer token IDs by the byte-level BPE tokenizer.
2. **Embedding:** token IDs are mapped to 1536-dimensional vectors. No position vector is added here; positions enter later through RoPE inside each attention layer.
3. **Layer transitions:** the vectors pass through 28 identical blocks. Each block applies RMSNorm, causal grouped query attention and a residual addition, then a second RMSNorm, the SwiGLU feed-forward network and another residual addition.
4. **Generation:** the final hidden state goes through a final RMSNorm and the language-model head to give logits over the vocabulary, which a softmax turns into probabilities. At T = 0.1 the distribution is sharply peaked, so decoding is close to greedy. The chosen token is appended, its keys and values are cached, and the loop repeats until the end-of-sequence token <|im_end|> or the 1024-token limit is reached.

## 4.5 Mathematical Formulation

**Causal grouped query self-attention**

For hidden states H from the previous layer, queries, keys and values are linear projections:

$$Q = HW_Q,\ K = HW_K,\ V = HW_V \tag{2}$$

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V \tag{3}$$

where M is the autoregressive mask that stops a position from attending to later positions:

$$M_{ij} = \begin{cases} 0, & if\ j \le i \\ -\infty, & if\ j > i \end{cases} \tag{4}$$

Because 12 query heads share 2 key/value heads, each KV head serves six query heads and the KV cache is six times smaller than with full multi-head attention. With 28 layers, head dimension 128 and FP16 storage, the cache holds 2 × 28 × 2 × 128 × 2 bytes, about 28 KB per token, or roughly 117 MB for a 4096-token context.

**Normalisation, feed-forward network and positions**

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \varepsilon}} \odot \gamma \tag{5}$$

$$\text{FFN}(x) = \left[\text{SiLU}\left(xW_{\text{gate}}\right) \odot \left(xW_{\text{up}}\right)\right]W_{\text{down}},\ \text{SiLU}(a) = a\,\sigma(a) \tag{6}$$

$$\theta_i = b^{-2i/d},\quad \langle R_m q, R_n k\rangle = q^T R_{n-m} k \tag{7}$$

In Equation 7 each pair of query/key dimensions is rotated by an angle proportional to its position, so the attention score depends only on the relative offset n - m between two tokens.

**Prompting formulations**

The five strategies differ in what is placed in the conditioning context. With instruction I, note X, K exemplar pairs and an intermediate rationale R:

$$\text{Zero-shot:}\quad P(Y \mid X, I) \tag{8}$$

$$\text{Few-shot:}\quad P(Y \mid X, I, (X_1, Y_1), \ldots, (X_K, Y_K)) \tag{9}$$

$$\text{Chain-of-thought:}\quad P(R \mid X, I) \cdot P(Y \mid X, R, I) \tag{10}$$

Von Oswald et al. [5] show that, for linear self-attention on regression tasks, a forward pass over in-context examples can emulate steps of gradient descent. Extending that picture to JSON extraction in a 1.5B language model is an interpretive hypothesis rather than something tested here, but it motivates why exemplars (Equation 9) can steer the output format without any weight update.

## 4.6 Why This Model Suits the Selected NLP Task

1. **Instruction following and structured output:** the Qwen2.5 series is reported to improve instruction following and the generation of structured output such as JSON [4], which is exactly what SIE requires.
2. **Small footprint:** at roughly 1 GB quantized, the model runs on ordinary laptops and workstations, including CPU-only machines, which keeps the study reproducible.
3. **Long native context:** the 32k-token native window leaves room for long notes plus few-shot exemplars (the runtime here was limited to 4096 tokens).
4. **Complete data privacy:** inference is fully on-premise, so no patient text is transmitted, which supports HIPAA and GDPR compliance goals.
5. **A demanding test bed:** because a 1.5B model is not reliably well-behaved, differences between prompt strategies are large enough to measure, which is the purpose of this study.

# 5. Dataset and Preprocessing

## 5.1 Dataset Description (Source, Size, Structure)

**Source and size.** The benchmark was authored by the team as **20 synthetic clinical encounter cases** plus **2 held-out exemplars** that are used only inside the Few-Shot and Chain-of-Thought prompts. All text is synthetic, so no real patient records or PHI are involved. Each case is a dictionary with an id, a category, the narrative text and a manually verified gold-standard ground_truth JSON object that follows the schema in Table 3. The cases span eight medical specialties (Table 5).

*Table 5: Specialties and example conditions in the benchmark dataset*

| Specialty | Example conditions |
|---|---|
| **Endocrinology** | Poorly controlled Type 2 diabetes mellitus with peripheral neuropathy |
| **Cardiology** | Acute decompensated heart failure; hypertensive urgency |
| **Pulmonology** | Community-acquired pneumonia; acute exacerbation of asthma |
| **Neurology** | Acute migraine with photophobia and phonophobia; transient ischemic attack |
| **Gastroenterology** | Acute appendicitis; symptomatic cholelithiasis (biliary colic) |
| **Rheumatology** | Seropositive rheumatoid arthritis with symmetric synovitis |
| **Dermatology** | Plaque psoriasis with erythematous scaly plaques |
| **Psychiatry / Infectious disease** | Major depressive disorder; acute streptococcal pharyngitis |

Each note contains a chief complaint, history of present illness, examination, diagnostic findings and a treatment plan, and deliberately includes negated findings (for example "denies fever"). The gold annotation (for a diabetic peripheral neuropathy case, the type used by test_pipeline.py) is:

```
{
 "patient_age": 62,
 "patient_gender": "Male",
 "primary_diagnosis": "Type 2 Diabetes Mellitus with Peripheral Neuropathy",
 "severity_level": "Moderate",
 "symptoms": ["bilateral burning foot pain", "paresthesias",
              "polyuria", "polydipsia"],
 "medications": [
   {"name": "Metformin", "dosage": "1000 mg", "frequency": "twice daily"},
   {"name": "Gabapentin", "dosage": "300 mg",
    "frequency": "nightly at bedtime"}
 ],
 "vitals_and_labs": {"HbA1c": "9.4%", "Fasting Blood Glucose": "198 mg/dL",
                     "Blood Pressure": "138/84 mmHg"},
 "follow_up_days": 30
}
```

**Evaluated subset.** run_experiment.py takes the first N cases in dataset order. The reported results use N = 8 (--num-cases 8), so the remaining twelve cases are available for extended runs.

## 5.2 Data Cleaning Steps

The text inputs need no cleaning, because the notes are authored. The data that does need cleaning is the model output. A self-healing parser (repair_json_string and extract_json_payload in src/prompt_templates.py) turns noisy generations into JSON objects in four steps:

1. **Fence stripping:** extract the interior of ``json ... or ... `` blocks.
2. **Boundary scanning:** keep only the text between the outermost { and }, discarding conversational commentary such as "Here is the requested JSON".
3. **Unquoted unit repair:** wrap number-plus-unit values in quotes with a regular expression (shown below).
4. **Trailing comma removal:** delete commas that precede a closing brace.

```
pattern : :\s*([0-9]+(?:\.[0-9]+)?\s+[a-zA-Z/%]+)\s*([,}])
replace : : "\1"\2
before  : "Fasting Blood Glucose": 198 mg/dL,
after   : "Fasting Blood Glucose": "198 mg/dL",
```

extract_json_payload returns two values: the parsed object and a flag saying whether the **raw** output parsed without any repair. The benchmark uses this flag as *strict validity*, so the repair stage can rescue an output for scoring without hiding the fact that the model itself failed. Note that the documented repair pattern targets the shape *number, space, unit* (such as 198 mg/dL); other malformed shapes, for example 138/84 mmHg, fall outside it and depend on the model's own compliance.

## 5.3 Tokenization, Lemmatization, Stopwords Removal

**Model side:** inference uses Qwen's native byte-level BPE tokenizer (about 151k tokens), so no tokenization is done by the pipeline itself. **Evaluation side:** normalize_text in src/evaluator.py lowercases text, replaces punctuation with spaces (re.sub(r"[^\w\s]", " ", text)) and collapses whitespace; entities are then compared as sets of distinct tokens. Lemmatization is not used: clinical terms are matched on normalised surface tokens with fuzzy overlap thresholds rather than morphological analysis. Because comparison is set-based, repeated words do not inflate overlap.

## 5.4 Train/Test Split Strategy

This study varies prompts and keeps model weights frozen, so there is no gradient training and no conventional train/validation split. Instead:

- **In-context set:** 2 cases are reserved as permanent exemplars for the Few-Shot and Chain-of-Thought prompts. They are excluded from the test cases, which prevents test-set leakage through the prompt.
- **Evaluation set:** 20 cases form the blind test pool; the results reported here use its first 8.

## 5.5 Feature Engineering or Embedding Approach

No hand-crafted features, TF-IDF vectors or static word embeddings are used. The model builds its own 1536-dimensional contextual representation at every layer, and the pipeline only sees the final text. Matching of extracted entities against the gold annotation uses token-overlap similarity with bipartite matching: a predicted entity counts as a match when its overlap with a gold entity reaches a threshold (0.35 for medications, 0.40 for symptoms). This lets "bilateral foot numbness" match "paresthesias of bilateral feet" without requiring identical wording.

# 6. Implementation Methodology

## 6.1 Tools, Libraries, and Environment Used

*Table 6: Software stack*

| Component | Role |
|---|---|
| **Python 3.11 (3.10+ supported)** | Language for the pipeline, evaluator, app and report generator |
| **Ollama + qwen2.5:1.5b** | Local inference server and quantized model (GGUF Q4_K_M) |
| **requests** | HTTP communication with the Ollama REST API |
| **Streamlit** | Interactive web application (app.py) |
| **Pandas** | Result aggregation (groupby, CSV export) and dashboard tables |
| **Matplotlib, Seaborn** | The five benchmark plots (src/visualize.py) |
| **python-docx** | Programmatic generation of report drafts (build_reports.py) |

The project is organised as follows:

```
NLP_AAT/
|-- src/
|   |-- dataset.py          # 20 benchmark cases + 2 exemplars
|   |-- prompt_templates.py  # 5 strategies, JSON extraction and repair
|   |-- llm_client.py        # LocalLLMClient: Ollama calls + latency/tokens
|   |-- evaluator.py         # StructuredIEEvaluator: validity, F1, negation
|   `-- visualize.py         # 5 publication-style charts (PNG)
|-- data/benchmark_cases.json
|-- results/                 # JSON run logs, CSV summary, plots/
|-- app.py                   # Streamlit dashboard
|-- run_experiment.py        # resumable CLI benchmark runner
|-- test_pipeline.py         # end-to-end sanity check
`-- README.md
```

## 6.2 Model Implementation Steps

1. **Environment setup:** pull and verify qwen2.5:1.5b in Ollama and start the server.
2. **Prompt formulation:** implement the five strategies in src/prompt_templates.py and register them in TECHNIQUE_REGISTRY, so the runner and the app share the same templates.
3. **Inference client:** implement LocalLLMClient in src/llm_client.py with check_health() and generate(), which returns the response text, latency, token counts and tokens per second.
4. **Evaluation engine:** implement StructuredIEEvaluator in src/evaluator.py to score validity, schema coverage, field-level F1 and negation violations.
5. **Sanity test:** run test_pipeline.py, which executes all five techniques on Case 001 and prints latency, parse status and metrics.
6. **Benchmark automation:** run run_experiment.py over the benchmark cases and write the JSON and CSV results.
7. **Visualisation and UI:** generate the plots with src/visualize.py and serve the dashboard with app.py.

The five strategies differ only in the prompt text (Table 7); everything downstream of the prompt is identical, which isolates the effect of prompt design.

*Table 7: The five prompt engineering strategies*

| Strategy | Registry key | Core idea | Prompt tokens |
|---|---|---|---|
| **Zero-Shot** | Zero-Shot | Direct instruction to extract the information; no schema, no examples | 247 |
| **Few-Shot ICL** | Few-Shot | Two held-out input-output demonstrations show the keys, nesting and quoting [1] | 971 |
| **Chain-of-Thought** | Chain-of-Thought | Model first reasons step by step about the note, including negations, then emits the JSON [2] | 948 |
| **Schema-Constrained** | Schema-Constrained | A formal schema with types, required keys and allowed values plus strict validity instructions | 534 |

| | | | |
|---|---|---|---|
| **Role-Expert** | Role-Expert | A "Principal Clinical Informatician" persona with explicit guardrails: no hallucination, negation filtering, medication decomposition | 607 |

## 6.3 API Integration

LocalLLMClient sends one HTTP POST per extraction to the Ollama generate endpoint with the payload below. The reply carries the generated text, from which the client returns response, latency_sec, prompt_tokens, completion_tokens, tokens_per_sec, plus success and error for failure handling.

```
POST http://127.0.0.1:11434/api/generate   (Content-Type: application/json)
{
  "model": "qwen2.5:1.5b",
  "prompt": "<formatted prompt text>",
  "stream": false,
  "options": { "temperature": 0.1, "top_p": 0.9, "num_predict": 1024 }
}
```

The commands used to run the system are:

```
ollama pull qwen2.5:1.5b && ollama serve
python test_pipeline.py                # sanity check on Case 001
python run_experiment.py --num-cases 8   # benchmark (--temperature 0.1)
python -m src.visualize                # regenerate the 5 plots
streamlit run app.py                   # dashboard at http://localhost:8501
```

## 6.4 Pseudocode / Workflow Diagram

Algorithm 1 describes how a single note is processed and scored.

```
Algorithm 1: Extraction and evaluation of one clinical note
Input : note X, strategy S, ground truth Y*
Output: parsed object Y_hat, metrics M
 1: prompt <- f_S(X)                    # template from TECHNIQUE_REGISTRY
 2: t0 <- now(); raw <- POST /api/generate(prompt, T=0.1); t1 <- now()
 3: latency <- t1 - t0
 4: (Y_hat, strict) <- extract_json_payload(raw)  # strict: no repair needed
 5: S_cov <- fraction of the 7 scored keys present in Y_hat
 6: F1_demo, F1_diag, F1_sev, F1_symp, F1_med, F1_follow <- scorers(Y_hat, Y*)
 7: nu <- negation_violation_rate(Y_hat.symptoms, X)
 8: F1_base <- 0.15 F1_demo + 0.25 F1_diag + 0.10 F1_sev
            + 0.25 F1_symp + 0.20 F1_med + 0.05 F1_follow
 9: F1_overall <- max(0, F1_base - 0.15 * nu)
10: return Y_hat, {strict, S_cov, field F1s, nu, F1_overall, latency}
```

Algorithm 2 is the benchmark runner (run_experiment.py), also shown as a flowchart in Figure 3. It checkpoints after every run, so an interrupted benchmark resumes where it stopped.

```
Algorithm 2: Resumable benchmark runner
Input : number of cases N, temperature T
 1: cases <- dataset[0 : N]; abort if the Ollama health check fails
 2: records <- load results/benchmark_results.json (if present)
 3: done <- {(case_id, technique) for each record}
 4: for each case c in cases:
 5:   for each (name, build_prompt) in TECHNIQUE_REGISTRY:
 6:     if (c.id, name) in done: continue          # skip finished work
 7:     res <- client.generate(build_prompt(c.text), T)
 8:     if not res.success: print error; continue    # nothing recorded
 9:     (obj, strict) <- extract_json_payload(res.response)  # {} on error
10:     m <- evaluator.evaluate_sample(obj, c.ground_truth, c.text)
11:     m.strict_json_valid <- strict              # raw validity
12:     append run record (metrics, latency, tokens, raw response)
13:     write benchmark_results.json               # checkpoint
14: summary <- groupby(technique).mean(metrics); sort by overall_f1
15: write results/summary_metrics.csv
```

Two design details matter for interpretation: calls that fail are not recorded, so a re-run retries only the missing pairs; and an output that cannot be parsed at all is scored as an empty object, so it contributes zero rather than being dropped from the averages.

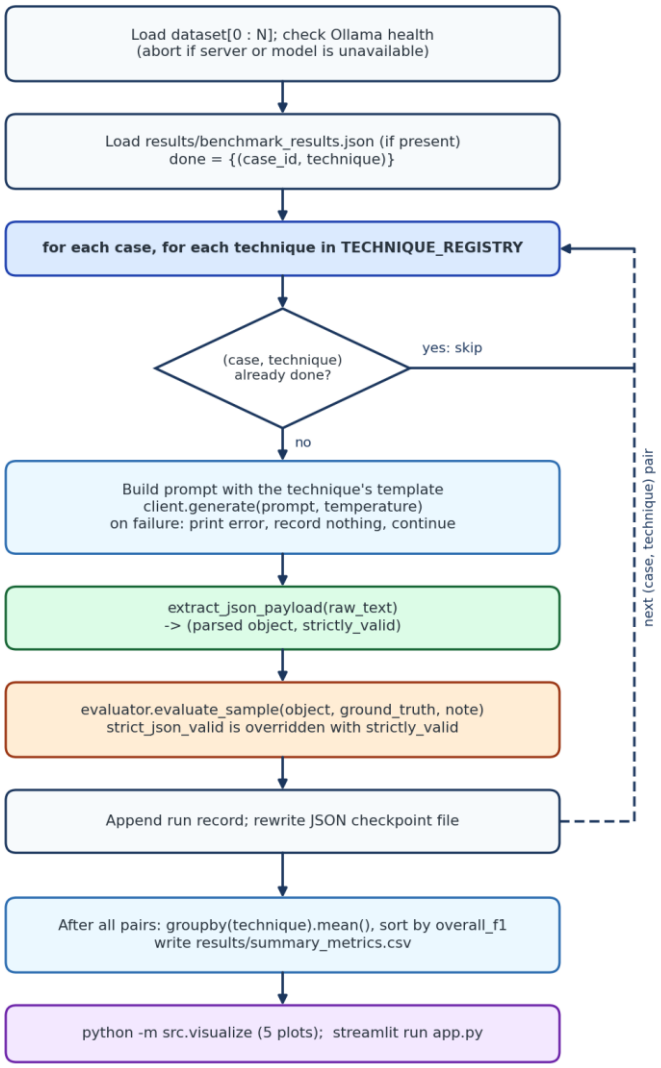



*Figure 3: Workflow of the resumable benchmark runner (run_experiment.py)*

## 6.5 Hyperparameters and Training Configuration

No training is performed: model weights are frozen and only the prompt changes. Decoding settings were identical for all five strategies.

*Table 8: Inference and experiment configuration*

| Setting | Value |
|---|---|
| **Model / quantization** | qwen2.5:1.5b, GGUF Q4_K_M |
| **Temperature T** | 0.1 (default of the CLI runner and of the app slider) |
| **Top-p (nucleus sampling)** | 0.9 |
| **Maximum new tokens (num_predict)** | 1024 |
| **Streaming** | Off (stream=False) |
| **Runtime context length** | 4096 tokens (model native limit: 32,768) |
| **Few-shot exemplars K** | 2 |
| **Strategies compared** | 5 |
| **Cases evaluated** | 8 of the 20 benchmark cases (first 8 in dataset order) |
| **Total inference runs** | 8 × 5 = 40, one run per (case, strategy) pair |

A temperature of 0.1 makes decoding near-deterministic but not bit-exact, and no random seed is set in the request, so repeating the benchmark can change individual outputs slightly.

# 7. Results and Evaluation

## 7.1 Performance Metrics

Table 9 lists the metrics. Accuracy is reported per field and as a weighted composite, alongside structural and cost metrics, because a strategy that is accurate but unparseable is not usable in practice.

*Table 9: Evaluation metrics*

| Metric | Definition | Range |
|---|---|---|
| **Strict JSON validity** | 1 if the raw output parses under RFC 8259 [14] before any repair, else 0 | {0, 1} |
| **Schema key coverage** | Share of the 7 scored keys present in the parsed output | [0, 1] |
| **Demographics F1** | Agreement of patient_age and patient_gender with ground truth | [0, 1] |
| **Diagnosis F1** | Token-overlap F1 of primary_diagnosis (Equation 12) | [0, 1] |
| **Severity F1** | Categorical agreement of severity_level | [0, 1] |
| **Symptoms P / R / F1** | Set-based overlap of symptom lists using bipartite matching (threshold 0.40) | [0, 1] |
| **Medications F1** | Set-based matching of medication entries (threshold 0.35) | [0, 1] |
| **Follow-up F1** | Agreement of follow_up_days with ground truth | [0, 1] |
| **Negation violation rate** | Fraction of extracted symptoms that match a negated finding in the note (Equation 13) | [0, 1] |
| **Overall F1** | Weighted composite minus a negation penalty (Equations 14 and 15) | [0, 1] |
| **Latency, tokens** | Wall-clock seconds per sample; prompt and completion tokens; tokens per second | n/a |

For a free-text field with predicted token set T-hat and gold token set T*, precision, recall and F1 are:

$$Prec = \frac{|\hat{T} \cap T^*|}{|\hat{T}|}, \quad Rec = \frac{|\hat{T} \cap T^*|}{|T^*|} \tag{11}$$

$$F1 = \frac{2 \cdot Prec \cdot Rec}{Prec + Rec} \tag{12}$$

The negation violation rate uses the set of extracted symptoms S-hat. Negation cues in the note follow the idea of NegEx-style rules [8]:

$$\nu = \frac{|\{s \in \hat{S} : s \text{ matches a negated finding in } X\}|}{|\hat{S}|} \tag{13}$$

The composite weights favour the clinically central fields (diagnosis, symptoms, medications), and negation errors are penalised directly:

$$\begin{aligned} F1_{\text{base}} = {} & 0.15\, F1_{\text{demo}} + 0.25\, F1_{\text{diag}} + 0.10\, F1_{\text{sev}} \\ & + 0.25\, F1_{\text{symp}} + 0.20\, F1_{\text{med}} + 0.05\, F1_{\text{follow}} \end{aligned} \tag{14}$$

$$F1_{\text{overall}} = max(0,\ F1_{\text{base}} - 0.15\, \nu) \tag{15}$$

These weights are a design choice made for this project and were not tuned on the data.

## 7.2 Visualization of Results

Five charts summarise the benchmark (n = 8 cases per strategy).

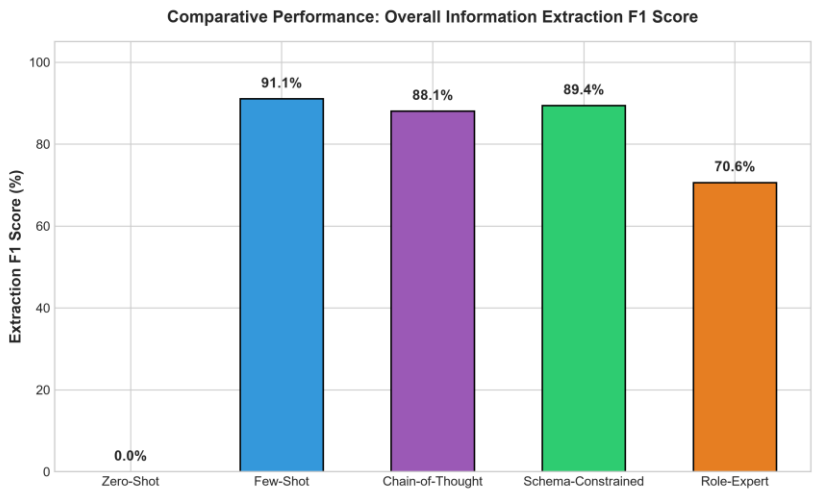



*Figure 4: Overall extraction F1 score by prompt strategy*

Figure 4 shows three strategies bunched at 88 to 91% and a clear gap to Role-Expert (70.6%). Zero-Shot scores zero because its output almost never uses the canonical keys.

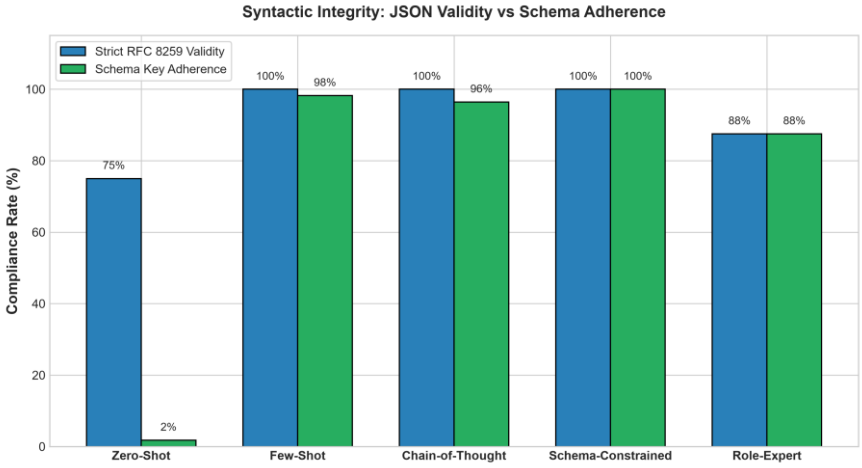



*Figure 5: Strict RFC 8259 validity versus schema key coverage*

Figure 5 separates two kinds of structural success. Zero-Shot outputs are often valid JSON (75%) but almost never follow the target schema (about 2% of keys), so its main failure is *wrong structure*, not broken syntax. Schema-Constrained is the only strategy at 100% on both bars.

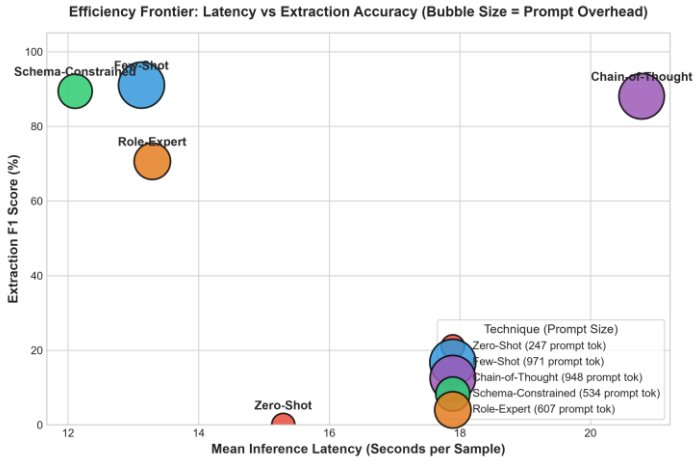



*Figure 6: Efficiency frontier: latency versus extraction accuracy (bubble size is prompt size)*

Figure 6 is the cost-accuracy view. Schema-Constrained and Few-Shot sit in the favourable upper-left region; Chain-of-Thought is as accurate but about 8 seconds slower.

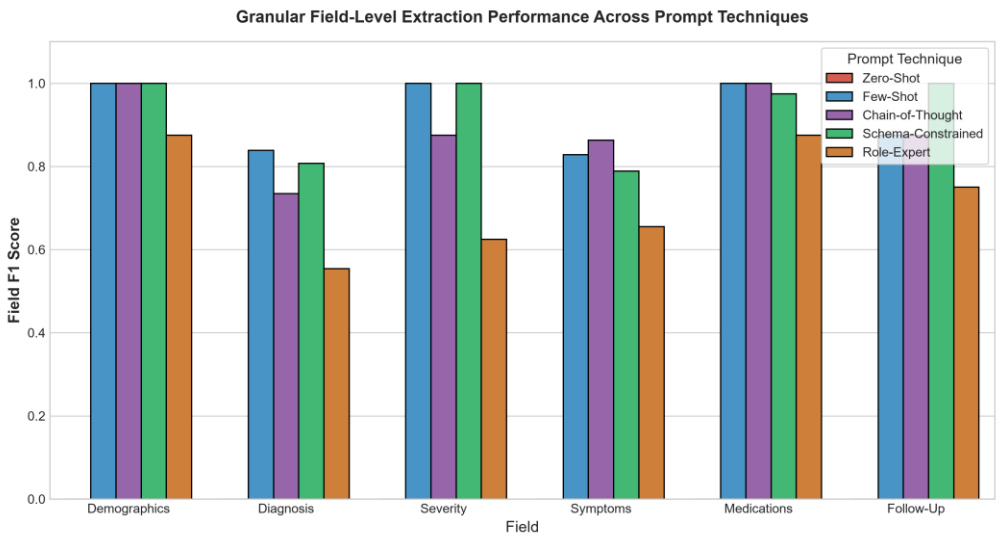



*Figure 7: Field-level F1 by prompt strategy*

Figure 7 shows where errors concentrate. Demographics and medications are near-perfect for the top three strategies, while diagnosis (55 to 84%) and symptoms (66 to 86%) are the hardest fields. The legend overlaps the follow-up bars in this chart, so those values are best read from the dashboard table.

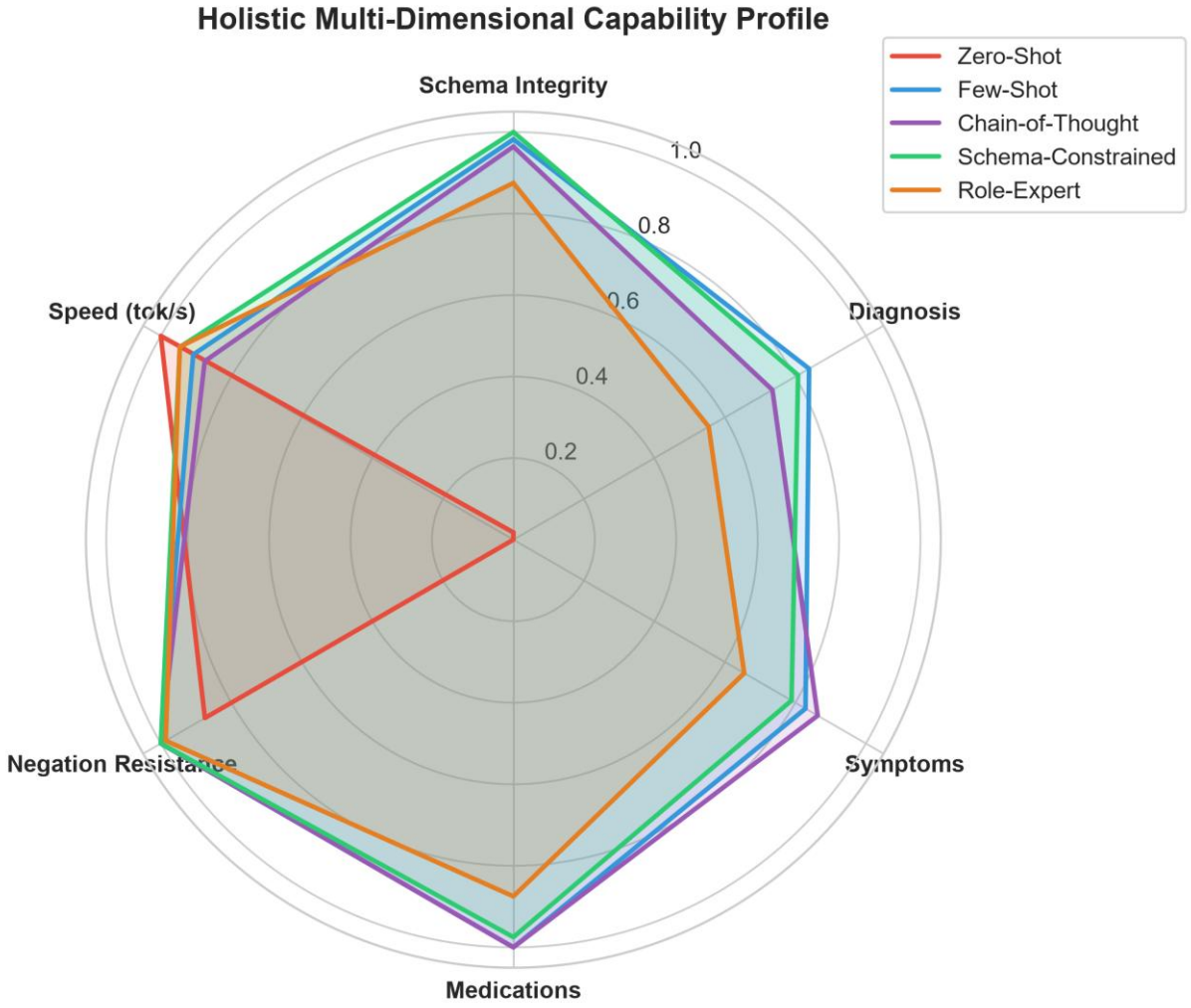



*Figure 8: Multi-criteria radar profile of the five strategies*

Figure 8 combines six axes. Zero-Shot looks strong on speed (tokens per second) and, misleadingly, on negation resistance, because it produced almost no canonical symptoms field to be penalised; its unconstrained answers tend to be long, which is consistent with its wall-clock latency not being the lowest.

## 7.3 Qualitative and Quantitative Analysis

Table 10 gives the aggregated results (means over 8 cases; best value per column highlighted).

*Table 10: Benchmark results by prompt strategy (n = 8 cases per strategy)*

| Strategy | Strict valid | Schema cov. | Diag F1 | Symp F1 | Meds F1 | Overall F1 | Neg. viol. | Latency |
|---|---|---|---|---|---|---|---|---|
| **Few-Shot ICL** | **100.0%** | 98.2% | **83.9%** | 82.9% | **100.0%** | **91.1%** | **0.0%** | 13.1 s |
| **Schema-Constrained** | **100.0%** | **100.0%** | 80.8% | 78.9% | 97.5% | 89.4% | **0.0%** | **12.1 s** |
| **Chain-of-Thought** | **100.0%** | 96.4% | 73.5% | **86.3%** | **100.0%** | 88.1% | **0.0%** | 20.8 s |
| **Role-Expert** | 87.5% | 87.5% | 55.4% | 65.5% | 87.5% | 70.6% | 1.4% | 13.3 s |
| **Zero-Shot †** | 75.0% | 1.8% | 0.0% | 0.0% | 0.0% | 0.0% | 12.5% | 15.3 s |

† Zero-Shot is scored against the canonical schema although it is never told that schema, so its zeros mostly measure schema discovery rather than clinical understanding. Its 12.5% negation rate comes from the very few runs that emitted a symptoms field at all.

### Key quantitative findings

- **Few-Shot ICL** has the highest overall F1 (91.1%) and the best diagnosis F1 (83.9%), with perfect medication F1.
- **Schema-Constrained** is the only strategy with 100% strict validity and 100% schema coverage, reaches 89.4% overall F1 and is the fastest (12.1 s) with a prompt of 534 tokens.
- **Chain-of-Thought** has the best symptoms F1 (86.3%) but is about 72% slower than Schema-Constrained (20.8 s versus 12.1 s).
- **Role-Expert** is unreliable in this setup: 87.5% validity and the lowest diagnosis and symptoms F1 among the working strategies.
- **Zero-Shot** fails structurally: 1.8% schema coverage and 0.0% overall F1.
- The top three strategies are within 3 points of overall F1. With 8 cases, one case moves a mean by up to 12.5 points, so these differences are small relative to the noise and should be read as a tie on accuracy; the separation is in cost and robustness.

*Table 11: Prompt size and latency by strategy*

| Strategy | Prompt tokens | × Zero-Shot | Latency (s) | Latency vs Schema-Constrained |
|---|---|---|---|---|
| **Zero-Shot** | 247 | 1.0× | 15.3 | +26% |
| **Few-Shot ICL** | 971 | 3.9× | 13.1 | +8% |
| **Chain-of-Thought** | 948 | 3.8× | 20.8 | +72% |
| **Schema-Constrained** | 534 | 2.2× | 12.1 | baseline |
| **Role-Expert** | 607 | 2.5× | 13.3 | +10% |

Latency tracks generated output more than prompt length. Few-Shot has 82% more prompt tokens than Schema-Constrained (971 versus 534) yet is only about 1 s slower, whereas Chain-of-Thought, with almost the same prompt size as Few-Shot, is about 8 s slower, consistent with the extra reasoning text it generates.

## 7.4 Error Analysis

### Zero-Shot: wrong structure more than broken syntax

Six of eight Zero-Shot outputs (75%) were strictly valid JSON, yet only 1.8% of the expected keys appeared: the model invented its own hierarchy (for example nesting data under a patient object or using names such as patient_symptoms). The other two outputs (25%) were not strictly valid, presumably from syntax slips such as unquoted numeric units. Both failure modes make automated database ingestion impossible without a schema in the prompt.

### Role-Expert: one unusable run and weaker content

Strict validity and schema coverage are both 87.5% (7 of 8, and 49 of 56 keys), consistent with one run producing no usable output. Its content errors are also larger than for the other working strategies (diagnosis 55.4%, severity about

63% and symptoms 65.5%). The persona-heavy prompt appears to encourage extra clinical commentary around or inside the JSON, which hurts parsing and precision. In this run the added persona and guardrails did not outperform the plain schema prompt.

**Chain-of-Thought and Few-Shot: cost and small gaps**

Chain-of-Thought reached 100% strict validity but 96.4% coverage (2 missing keys in 56), and its diagnosis F1 (73.5%) is below that of Few-Shot and Schema-Constrained. Its main cost is the longer generation. Few-Shot missed a single key across the benchmark (98.2%), the only structural slip among the exemplar-based and schema-based strategies.

**Diagnosis is the hardest field**

For every working strategy the diagnosis field scores lowest (55 to 84%) while medications score 88 to 100%. Diagnosis is free text scored by token overlap, so a clinically equivalent but differently worded diagnosis loses credit, whereas medication entries have a more constrained form. Part of this gap is therefore a property of the metric, not only of the model.

> **Check against raw outputs.** Descriptions of specific failure patterns above (invented key names, extra commentary) should be confirmed against the raw_response and parse_error fields in results/benchmark_results.json; the Case-Level Breakdown tab of the dashboard lists them per run.

## 7.5 Comparison Between Approaches

*Table 12: Best strategy by criterion*

| Criterion | Best strategy | Value |
|---|---|---|
| **Overall extraction F1** | Few-Shot ICL | 91.1% |
| **Structural reliability (validity and coverage)** | Schema-Constrained | 100.0% and 100.0% |
| **Speed** | Schema-Constrained | 12.1 s per sample |
| **Prompt cost among usable strategies** | Schema-Constrained | 534 tokens |
| **Symptoms F1** | Chain-of-Thought | 86.3% |
| **Negation safety** | Few-Shot, Schema, CoT (tie) | 0.0% violations |

Few-Shot ICL leads Schema-Constrained by 1.7 points of overall F1 (91.1% versus 89.4%), but Schema-Constrained needs 45% fewer prompt tokens (534 versus 971), is about 1 s faster, and is the only strategy with perfect structure. Because the accuracy gap is well inside the sampling noise of an 8-case benchmark, Schema-Constrained is the best overall trade-off for deployment, while Few-Shot is a strong choice when prompt length is not a constraint.

Schema-Constrained matches or beats Chain-of-Thought on overall F1, diagnosis F1, schema coverage and latency, while Chain-of-Thought leads on symptoms and medications F1. Role-Expert and Zero-Shot are worse than Schema-Constrained on accuracy, validity, coverage and latency alike.

# 8. Discussion

## 8.1 Interpretation of Results

- **Structure is the first bottleneck.** The 1.5B model has enough clinical knowledge to fill most fields correctly once the format is specified (Few-Shot, Schema-Constrained and CoT all exceed 88% overall F1), but without a specification it does not produce the target structure at all.
- **Two ways to supply structure work.** Exemplars (Few-Shot) and an explicit schema (Schema-Constrained) give almost the same accuracy; the schema does so with about half the prompt tokens.
- **Reasoning helps one field, at a price.** Chain-of-Thought gives the best symptoms F1, but Few-Shot and Schema-Constrained also reach 0.0% negation violations, so these data do not show that reasoning is uniquely good at negation; its benefit is small relative to a 72% latency increase.
- **Persona is not a substitute for structure.** Role-Expert added a longer prompt and guardrails yet trailed Schema-Constrained by about 19 points of overall F1 in this run.

All of this holds for 8 cases, one model and one decoding setting, so the ranking of the top three should be treated as indicative, not conclusive.

## 8.2 Strengths and Weaknesses of the Model

**Strengths**

- Fast and cheap: about 12 to 13 s per note for the best strategies on local hardware, with no API fees.
- Strong adherence to a clear schema or examples (100% strict validity for three strategies).
- Complete data privacy: nothing leaves the machine.

**Weaknesses**

- High sensitivity to the prompt: the same model ranges from 0.0% to 91.1% overall F1 depending only on prompt design.
- Syntax and structure failures under open-ended instructions, and no hard validity guarantee.
- Weaker on free-text fields such as the diagnosis wording than on constrained fields.
- Limited context in the 4096-token runtime configuration used here, which restricts very long notes.

## 8.3 Challenges Encountered During Implementation

- **Repairing JSON without corrupting it:** the regular expression for unquoted units must not alter text inside valid string fields, and some malformed shapes (such as 138/84 mmHg) fall outside it.
- **Keeping repair from masking failure:** strict validity is measured before repair so that the self-healing step cannot inflate the metric.
- **Fair entity matching:** exact string comparison would punish clinically equivalent wording, so thresholded bipartite token-overlap matching was designed for symptoms and medications.
- **Negation handling:** deciding whether an extracted symptom is a negated finding requires cue-based rules in the evaluator [8].
- **Inference time:** the five strategies together take about 75 s of generation per case (13.1 + 12.1 + 20.8 + 13.3 + 15.3 s), so the 8-case benchmark is about 10 minutes of pure inference and all 20 cases would take about 25 minutes. A resumable runner with incremental checkpoints was built for this reason.

## 8.4 Impact of Dataset Quality and Preprocessing

The authored cases include explicit negations and multiple medications per note, which exposed failure modes (negated symptoms extracted as active, off-schema keys) that simple extraction tests miss. The repair stage lets

outputs with fixable syntax errors still be scored, while the separate strict-validity metric keeps the model's own reliability visible. The following limitations apply to the conclusions:

- **Small sample:** 8 cases per strategy; one case shifts a mean by up to 12.5 points, and no confidence intervals are computed.
- **Single run per pair:** no repeated runs, so run-to-run variation at T = 0.1 is not quantified.
- **Synthetic text:** authored notes are cleaner and more explicit than real clinical documentation.
- **Metric design:** token-overlap F1 is lenient on wording, and the composite weights are not tuned.
- **Single model:** conclusions may not transfer to larger models or other model families.

## 8.5 How API or Application Enhances the NLP Task

- **Accessibility:** clinicians and researchers can try any technique on any note without writing code.
- **Transparency:** the Executed Prompt and Raw Model Generation tabs show exactly what the model saw and produced, which makes failures easy to diagnose.
- **Immediate feedback:** latency, token counts, JSON status and ground-truth scores appear alongside each extraction, and side-by-side mode makes strategy trade-offs concrete.
- **Reproducibility:** the dashboard reads the same result files the CLI runner writes, so the figures in this report can be explored interactively.
- **Privacy by design:** the local Ollama API means experiments can be run on realistic notes without any cloud transfer.

# 9. Conclusion

## 9.1 Summary of the Work Done

This project implemented and compared five prompt engineering strategies for structured clinical information extraction with a local Qwen2.5:1.5B-Instruct model served by Ollama. We built a multi-criteria evaluation engine, a resumable benchmark runner, five result charts and an interactive Streamlit application, and benchmarked the strategies on 8 of the 20 curated clinical cases (40 inference runs).

## 9.2 Key Learnings and Insights

1. **Zero-Shot prompting is unsuitable** for this model and task: it produced valid-looking JSON with the wrong structure (1.8% schema coverage, 0.0% overall F1).
2. **Schema-Constrained prompting gives the best overall trade-off:** 100% strict validity and coverage, 89.4% F1, the lowest latency (12.1 s) and a prompt of 534 tokens.
3. **Few-Shot ICL reaches the highest F1 (91.1%)** but uses 82% more prompt tokens than the schema prompt for a gain that is within sampling noise at this sample size.
4. **Chain-of-Thought improves symptom extraction (86.3% F1)** at a cost of about 72% more latency.
5. **Prompt structure matters more than persona:** the role-based prompt with guardrails did not beat the plain schema prompt.
6. **Report limitations with results:** with 8 cases, one case moves a mean by 12.5 points, so small differences should not be over-interpreted.

## 9.3 Real-World Applicability

The local architecture is a practical starting point for privacy-preserving EHR ingestion, clinical decision support and coding assistance, with no cloud dependency and no per-call cost. Before any clinical use it would need validation on real de-identified notes, a larger test set, schema validation of every output, and human review of the extracted fields.

## 9.4 Future Improvements or Extensions

- **Constrained decoding:** enforce the schema at the sampling level (GBNF grammars, Outlines, or Ollama's JSON-schema output option) to guarantee valid JSON.
- **Stronger evaluation:** run all 20 cases, repeat runs and report confidence intervals, and add semantic matching for diagnoses.
- **Hybrid prompts:** combine the schema with one or two exemplars and a short, bounded reasoning step.
- **Negation post-filtering:** add a NegEx-style rule filter [8] after generation.
- **Fine-tuning:** parameter-efficient fine-tuning (LoRA) on de-identified clinical corpora such as MIMIC-IV.
- **Larger models:** evaluate 3B, 7B and 14B models on GPU hardware to see how the ranking changes.

# 10. References


**[1]** Brown, T., Mann, B., Ryder, N., et al. (2020). *Language Models are Few-Shot Learners*. Advances in Neural Information Processing Systems (NeurIPS), 33, 1877–1901.

**[2]** Wei, J., Wang, X., Schuurmans, D., et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. NeurIPS, 35, 24824–24837.

**[3]** Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). *Attention Is All You Need*. NeurIPS, 30, 5998–6008.

**[4]** Qwen Team (Yang, A., et al.). (2024). *Qwen2.5 Technical Report*. arXiv:2412.15115.

**[5]** von Oswald, J., Niklasson, E., Randazzo, E., et al. (2023). *Transformers Learn In-Context by Gradient Descent*. Proceedings of the 40th International Conference on Machine Learning (ICML), PMLR 202, 35151–35174.

**[6]** Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. Proceedings of NAACL-HLT, 4171–4186.

**[7]** HL7 International. (2022). *Fast Healthcare Interoperability Resources (FHIR), Release 4B (v4.3.0)*. https://hl7.org/fhir/R4B/

**[8]** Chapman, W. W., Bridewell, W., Hanbury, P., et al. (2001). *A simple algorithm for identifying negated findings and diseases in discharge summaries*. Journal of Biomedical Informatics, 34(5), 301–310.

**[9]** Ainslie, J., Lee-Thorp, J., de Jong, M., et al. (2023). *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. Proceedings of EMNLP 2023.

**[10]** Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., & Liu, Y. (2024). *RoFormer: Enhanced Transformer with Rotary Position Embedding*. Neurocomputing, 568, 127063.

**[11]** Zhang, B., & Sennrich, R. (2019). *Root Mean Square Layer Normalization*. NeurIPS, 32.

**[12]** Shazeer, N. (2020). *GLU Variants Improve Transformer*. arXiv:2002.05202.

**[13]** Rafailov, R., Sharma, A., Mitchell, E., et al. (2023). *Direct Preference Optimization: Your Language Model Is Secretly a Reward Model*. NeurIPS, 36.

**[14]** Bray, T. (Ed.). (2017). *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC 8259, Internet Engineering Task Force.

**[15]** Ollama. *Ollama: run large language models locally*. https://ollama.com

**[16]** Streamlit Inc. *Streamlit: a faster way to build and share data apps*. https://streamlit.io